

LUIS FERNANDO ALMEIDA DE OLIVEIRA

**A IMPORTÂNCIA DA ENGENHARIA DE SOFTWARE NA MANUTENÇÃO DE
SISTEMAS:
UM ESTUDO REFLEXIVO SOBRE DEAD BY DAYLIGHT E A BEHAVIOUR
INTERACTIVE**

São Paulo
2026

LUIS FERNANDO ALMEIDA DE OLIVEIRA

**A IMPORTÂNCIA DA ENGENHARIA DE SOFTWARE NA MANUTENÇÃO DE
SISTEMAS:
UM ESTUDO REFLEXIVO SOBRE DEAD BY DAYLIGHT E A BEHAVIOUR
INTERACTIVE**

Documentação acadêmica elaborada com o objetivo de analisar a importância da Engenharia de Software, da manutenção contínua e do ambiente organizacional no desenvolvimento de produtos digitais, utilizando Dead by Daylight, da Behaviour Interactive, como estudo de caso.

São Paulo
2026

RESUMO

Este trabalho apresenta uma reflexão sobre a importância da Engenharia de Software na construção, evolução e manutenção de sistemas de longa duração, tomando como foco Dead by Daylight (DBD), desenvolvido pela Behaviour Interactive (BHVR). O jogo completou dez anos em 2026 e evoluiu continuamente por meio de novos personagens, vantagens, mapas, modos e integrações. Essa longevidade evidencia desafios comuns a sistemas de software: código legado, dívida técnica, regressões, testes, manutenção evolutiva e necessidade de planejamento escalável. O estudo também aborda o papel do ambiente de trabalho, da comunicação entre equipes e da estabilidade organizacional na capacidade de manter conhecimento técnico e qualidade ao longo do tempo.

Palavras-chave: Engenharia de Software. Manutenção de Software. Código Legado. Dívida Técnica. Dead by Daylight. Behaviour Interactive.

1 INTRODUÇÃO

Engenharia de Software não se limita à implementação de funcionalidades. Ela envolve requisitos, arquitetura, qualidade, testes, manutenção, documentação, gestão de mudanças, comunicação e planejamento do ciclo de vida do produto. Esses elementos tornam-se ainda mais importantes em sistemas que permanecem ativos por muitos anos e precisam evoluir sem interromper o serviço oferecido aos usuários.

Dead by Daylight é um exemplo relevante. Lançado em 2016, o jogo completou dez anos em 2026 e a própria Behaviour Interactive apresentou planos para continuar evoluindo a base atual durante a próxima década, em vez de simplesmente substituí-la por uma sequência. Esse cenário transforma a manutenção em uma decisão estratégica, e não apenas em uma atividade corretiva (BEHAVIOUR INTERACTIVE, 2026a).

2 JUSTIFICATIVA DO ESTUDO DE CASO

A escolha da Behaviour Interactive e de Dead by Daylight como exemplo também possui uma motivação pessoal e prática. O autor deste trabalho é consumidor do produto e acompanha o jogo como jogador. A experiência de observar atualizações,

correções, regressões e comunicados da desenvolvedora despertou o interesse em analisar o mesmo produto sob a perspectiva da Engenharia de Software. Assim, um problema inicialmente percebido como usuário passa a ser interpretado por conceitos como manutenção, escalabilidade, arquitetura, dívida técnica, testes e gestão de equipes.

3 LONGEVIDADE, COMPLEXIDADE E CÓDIGO LEGADO

Um software mantido durante dez anos inevitavelmente acumula decisões técnicas de diferentes épocas. Código legado não significa necessariamente código de baixa qualidade; significa, sobretudo, código existente que continua sustentando o produto e cuja alteração pode possuir impactos difíceis de prever. Em um jogo como DBD, essa dificuldade aumenta porque novos killers, survivors, perks, itens, mapas, eventos e modos precisam coexistir com sistemas criados muitos anos antes.

A própria Behaviour reconhece publicamente essa complexidade. Em 2026, uma atualização alterou a forma como modificadores de gameplay se acumulam e a empresa afirmou que, após dez anos, as combinações haviam se tornado cada vez mais intensas. A mudança também foi apresentada como uma forma de permitir a criação de novos poderes, itens e vantagens com menor risco de gerar situações indesejadas ou quebradas (BEHAVIOUR INTERACTIVE, 2026b).

Em entrevista de 2026, o diretor criativo sênior Dave Richard relatou ainda que, nos primeiros anos, existia uma brincadeira interna de que qualquer correção parecia quebrar as bear traps em algum outro ponto. Segundo ele, os processos e a equipe amadureceram muito ao longo da década, mas tecnologias antigas ainda permanecem e estão sendo atualizadas. O relato é uma demonstração direta do problema de regressão em sistemas complexos (RESTART.RUN, 2026).

4 DÍVIDA TÉCNICA E O CICLO DE MANUTENÇÃO

A dívida técnica pode ser compreendida como o custo futuro criado quando soluções, estruturas ou decisões que atendem às necessidades atuais passam a dificultar mudanças posteriores. Em um live-service, esse custo pode crescer continuamente, pois o produto não para enquanto a equipe realiza manutenção.

No caso analisado, forma-se um risco de ciclo: novas funcionalidades são adicionadas; surgem incompatibilidades ou regressões; a equipe utiliza parte de sua capacidade para investigar e corrigir os problemas; simultaneamente, novas atualizações precisam continuar sendo produzidas. Se a modernização estrutural não recebe espaço suficiente, mais funcionalidades são adicionadas sobre sistemas antigos, aumentando novamente a superfície de manutenção.

Isso ajuda a explicar por que a presença de bugs não deve ser automaticamente interpretada como incapacidade dos programadores. A vaga pública de Gameplay Programmer da Behaviour, por exemplo, descreve trabalho conjunto entre programação, design, arte, animação e QA, além da responsabilidade de produzir código claro, manutenível e documentado. O desafio está em aplicar essas práticas continuamente dentro de uma base extremamente extensa e ativa (BEHAVIOUR INTERACTIVE, 2026c).

5 MANUTENÇÃO COMO PARTE DA ENGENHARIA DE SOFTWARE

A manutenção precisa ser planejada desde o desenvolvimento e permanecer como atividade permanente. Ela não se resume a corrigir defeitos depois que chegam ao usuário. Inclui manutenção corretiva, adaptativa, evolutiva e preventiva. A preventiva é especialmente importante nesse caso: refatoração, redução de acoplamento, remoção de componentes obsoletos, documentação, melhoria da observabilidade e expansão de testes automatizados diminuem o risco das próximas mudanças.

Para DBD, uma estratégia tecnicamente mais sustentável seria modernizar a aplicação de forma incremental. Uma equipe ou capacidade dedicada poderia trabalhar nos componentes mais críticos enquanto o restante da organização mantém capítulos, eventos e atualizações. Sistemas novos seriam validados em paralelo e substituiriam gradualmente implementações antigas. Essa abordagem evita a necessidade de descartar de uma só vez um produto com uma década de conteúdo e uma base ativa de usuários.

A decisão da Behaviour de continuar evoluindo a fundação atual do jogo torna essa necessidade ainda mais relevante. No aniversário de dez anos, a empresa anunciou que pretende investir na evolução do DBD existente e apresentou uma grande

reformulação visual para 2027, além de outras mudanças futuras (BEHAVIOUR INTERACTIVE, 2026a).

6 AMBIENTE DE TRABALHO, COMUNICAÇÃO E CONHECIMENTO

A qualidade de software também depende do ambiente em que ele é produzido. Sistemas antigos carregam conhecimento que muitas vezes não está completamente registrado em documentos: decisões arquiteturais, limitações históricas, comportamentos específicos e motivos pelos quais determinadas soluções existem. Por isso, retenção de profissionais, comunicação, documentação e transferência de conhecimento são elementos técnicos, além de organizacionais.

A Behaviour atualmente apresenta oficialmente um modelo híbrido, com trabalho remoto às segundas e sextas-feiras e colaboração presencial de terça a quinta-feira (BEHAVIOUR INTERACTIVE, 2026d). Independentemente da preferência por remoto, híbrido ou presencial, mudanças de política precisam ser comunicadas com previsibilidade. Um ambiente de confiança facilita retenção e cooperação; a saída frequente de pessoas experientes pode aumentar o custo de manutenção de uma base antiga porque parte do conhecimento histórico também deixa a equipe.

Da mesma maneira, a integração entre programação, QA, design, arte, produção e áreas de negócio é indispensável. Uma funcionalidade não termina quando o código compila: precisa ser testada, integrada, comunicada, monitorada e mantida. A Engenharia de Software oferece processos para que essa colaboração não dependa apenas do esforço individual dos profissionais.

7 ESCALABILIDADE E UMA PROPOSTA DE EVOLUÇÃO

Pensar de forma escalável significa projetar não apenas para a próxima atualização, mas para a capacidade de realizar centenas de atualizações futuras. No contexto de DBD, isso implica identificar módulos com maior incidência de regressões, estabelecer métricas de qualidade, fortalecer testes automatizados, criar contratos claros entre sistemas, documentar decisões arquiteturais e reservar capacidade permanente para refatoração.

A modernização não precisa ocorrer como um 'Dead by Daylight 2'. Ela pode ser progressiva: mapear dependências; priorizar componentes críticos; criar testes que preservem comportamentos esperados; refatorar ou substituir módulos; observar o resultado em produção; remover o legado quando a nova implementação estiver estável; e repetir o processo. Essa estratégia transforma manutenção em fluxo contínuo de engenharia.

O fato de a própria equipe reconhecer que ainda existem tecnologias antigas sendo atualizadas indica que parte desse processo já ocorre. Porém, considerando a intenção pública de sustentar DBD por mais uma década, a modernização técnica precisa acompanhar a mesma ambição de produto (RESTART.RUN, 2026; BEHAVIOUR INTERACTIVE, 2026a).

8 CONSIDERAÇÕES FINAIS

O caso de Dead by Daylight demonstra por que Engenharia de Software é essencial em ambientes reais. Um produto pode ser comercialmente bem-sucedido e continuar exigindo decisões difíceis sobre arquitetura, manutenção, testes, pessoas e evolução. Bugs recorrentes não devem ser analisados apenas como falhas isoladas de implementação: podem representar a consequência acumulada da complexidade de um sistema que cresce continuamente.

Se a Behaviour tivesse conseguido, desde o início, prever a escala que Dead by Daylight alcançaria e mantido um ambiente contínuo de atualização arquitetural, refatoração e manutenção preventiva, provavelmente parte da dificuldade atual seria menor. Naturalmente, o sucesso e a longevidade do jogo eram difíceis de prever em 2016. Agora, porém, essa justificativa deixa de existir: a empresa sabe que pretende sustentar o produto por muitos outros anos. Por isso, o processo de manutenção estrutural precisa começar ou ser ampliado com urgência e tratado como prioridade estratégica da BHVR. Continuar adicionando conteúdo sem reservar capacidade suficiente para modernizar a base apenas aumenta o custo futuro. Pensar de forma escalável, preservar conhecimento técnico e manter um processo permanente de manutenção são justamente exemplos de como a Engenharia de Software deixa de ser teoria e passa a determinar a sustentabilidade de um produto real.

REFERÊNCIAS

BEHAVIOUR INTERACTIVE. Dead by Daylight 10th Anniversary Recap: Announcements & Celebrations. 15 jun. 2026. Disponível em: deadbydaylight.com/news/dbd-10th-anniversary-recap-announcements-celebrations/. Acesso em: 2 set. 2026.

BEHAVIOUR INTERACTIVE. 9.6.0 | PTB. 7 abr. 2026. Disponível em: support.deadbydaylight.com/hc/pt-br/articles/47979349838740-9-6-0-PTB. Acesso em: 2 set. 2026.

BEHAVIOUR INTERACTIVE. Gameplay Programmer - Dead by Daylight. 2026. Disponível em: bhvr.com/poste/gameplay-programmer-dead-by-daylight-3/. Acesso em: 2 set. 2026.

BEHAVIOUR INTERACTIVE. Jobs & Career Opportunities. 2026. Disponível em: bhvr.com/jobs/. Acesso em: 2 set. 2026.

RESTART.RUN. Behaviour Discusses Dead by Daylight's Overhaul, Future Plans, and a Decade of Development. 18 jun. 2026. Disponível em: restart.run/articles/dead-by-daylight-future-plans-interview-with-behaviour. Acesso em: 2 set. 2026.